SPE Report

CrowdVision — Software Process Engineering Report

Authors: Ettore Farinelli · Nicolò Ghignatti · Mounir Samite **Course:** Software Process Engineering · **Date:** June 2026

CrowdVision is a **digital-twin and crowd-analytics platform**. It takes live data from sensors placed around a building and renders it on an interactive 3D model, so whoever runs the place can see — in real time — how crowded a room is, how warm it is, and whether something needs attention. It is meant to scale from a single floor up to a whole campus.

⚠ This is a short, high-level report — not the documentation

This document is deliberately **light**. It is a *summary* written for the Software Process Engineering exam, not the project's real documentation. We only sketch each topic here; the actual detail — every domain model, every diagram, every workflow — lives in the **Developer Guide**. Whenever a topic is worth a closer look, a **Read more** box at the end of the section points to the exact page (and often the exact section) that covers it properly. If you want the full picture, please follow those links rather than relying on this report alone.

What follows is therefore less a tour of the features and more an account of *how we built it*: how we split the problem into a domain model, the architecture we settled on, and the tooling that

1 · Introduction

We built CrowdVision **domain-first**: we modelled the problem before deciding how to run it. As Section 2 explains, the domain splits naturally into several independent parts, and in our scenario it was convenient to give each part its own **service**. The result is a **polyglot, multi-service system** living in a single **monorepo** — every service, the shared tooling, the infrastructure manifests and the documentation sit in one Git repository. Concretely that is ten deployable pieces: seven backend services, two device simulators, and a Vue single-page app for the browser.

1.1 What we set out to build

The goals below are about *the product* — what CrowdVision is supposed to do. The constraints the exam itself imposes are a separate matter, handled in Section 1.2 below. As a product, we wanted CrowdVision to:

- **Ingest live telemetry at scale.** A building full of sensors emits a constant stream of readings — people counts, temperature, and so on — and the system had to take that stream in continuously, not in batches.
- **Be a genuinely data-intensive application.** The interesting engineering is in moving and shaping data: ingesting it, checking it against thresholds, filtering it per building, and fanning it out to whoever is watching — all in real time.
- **Render the building, live, on the front page.** The centrepiece is an interactive **3D model** of the building that updates as readings arrive: a room visibly fills up or warms up as it happens. This is the product’s differentiator, and where we spent the most modelling effort.
- **Push updates to the browser in real time.** A reading taken by a sensor should reach the operator’s screen within moments, not on a manual refresh.
- **Alert on what matters.** When a reading crosses a threshold, the right people should be notified without anyone having to watch the dashboard.

Read more

The product scope in full: Functional Requirements · Non-Functional Requirements · User Stories.

1.2 The exam constraints, and how we met them

Software Process Engineering also sets out a handful of requirements that any project has to satisfy. They are not the same thing as the product goals above, so we list them explicitly here, each next to what we actually did to meet it.

Exam requirement	How we satisfied it
Domain-driven design	We started from a DDD strategic design: one domain, several subdomains, and bounded contexts joined by a context map (see Section 2.1).
A clear development process and DevOps practices	GitHub Flow on a single protected branch, Conventional Commits and a branch-naming convention, every change reviewed and gated by CI (see Sections 1.3 and 3.3).
Full-scale automation, including CI and CD	One CI gate fans out build / test / lint / audit / scan per service; `semantic-release` then versions and deploys every merge (see Section 3.4).
Deploy automation via containerization and/or orchestration	Every service ships as a Docker image; the system runs under Docker Compose locally and on Kubernetes in production (see Section 3.2).
Technically involve two or more target platforms	Our services run on three different runtimes — TypeScript on Node.js, Python, and Rust — clearing the “different

1.3 How we organised the work

Three habits carried the day-to-day:

- **Version control.** We work on **GitHub Flow**: `master` is the one protected branch, and nothing reaches it except through a reviewed pull request that passes the automated gate. Commit messages follow **Conventional Commits**, and branches follow a naming convention — both checked by a machine, not by trust.
- **Build & quality. moon** runs each project's `build` / `test` / `lint` / `audit` tasks, caches the results, and only re-runs what a change actually touched, so a one-line edit never rebuilds the whole repo. Every service brings its own tests, linting and dependency audit.
- **Release & deployment.** We don't pick version numbers — **semantic-release** reads them straight off the commit history. Every merge to `master` cuts a release, builds and scans the images, and pushes them to **GHCR**, which is exactly where production pulls from.

1.4 Licensing

CrowdVision is **proprietary, source-available software** — *"All rights reserved"*. The repository is public so anyone can read the code and judge it. GitHub's terms let people view and fork it on GitHub, but any fork stays under our licence — running it, building a product from it, redistributing it elsewhere or selling it is off the table without our say-so.

That is a conscious middle ground. Permissive licences (MIT, Apache-2.0) would let someone fold the code into a closed product; strong copyleft (GPL-3.0) would force every derivative to stay open. We wanted neither — we wanted the work to be **open to inspection while staying ours to govern**.

This openness is deliberately for *now*. We like having CrowdVision in the open, where it can be read and learned from, and we intend to keep it that way through this stage of the project. But the plan is for CrowdVision to grow into a product, so we have reserved the right to change the

Read more

The full licence terms live in the repository `LICENSE`.

---

2 · Design

We designed CrowdVision **domain-first**. Before settling on any technology, we used Domain-Driven Design to understand the problem and carve it into parts; only afterwards did we decide how to run those parts. We treated DDD as a **compass, not a rulebook**: the strategic boundaries are the part we were most confident in; tactical detail we added only where it paid for itself.

2.1 The domain, and how we split it

The domain, boiled down, is *watching physical spaces in real time through a digital twin*. It is not one undivided thing — it falls into several **subdomains**, each with its own language and its own job. We sorted them by how much they actually set the product apart, which in turn told us where to spend our modelling effort:

Subdomain (type)	Lives in
Spatial Digital Twin · *Core*	twin-service
Telemetry Ingestion & Thresholds · *Core*	sensor-service
Telemetry Distribution · *Core*	contracts-service
Identity & Access · *Supporting*	auth-service
Alerting · *Supporting*	notification-service

Real-Time Transport · *Generic*	socket-service

Every core and supporting subdomain became exactly one **bounded context**, and we made that boundary do double duty as a storage and deployment boundary too. The contexts talk to each other through the familiar DDD patterns: the signed **identity token is the Published Language** everyone agrees on, while the building-to-telemetry and building-to-alerting links are **Customer/Supplier** with an **Anti-Corruption Layer** on the receiving end so no model leaks across.

Read more

The full subdomain breakdown, context map, and the *refer-to-a-domain-by-name* convention: Strategic Design → Context Map. The per-context models: Identity & Access · Building Management · Telemetry Ingestion · Telemetry Distribution · Alerting.

2.2 Why each context became its own service

Nothing about DDD forces a bounded context to be a separate deployable — a context can live perfectly well as a module inside one larger application. In our scenario, though, it was convenient to give each context **its own service**: the parts have genuinely different workloads (a Rust telemetry filter has little in common with a Python assistant), several of them need to scale on their own, and a process boundary is the most honest way to stop one context's model leaking into another's. So we mapped each context onto exactly one deployable service. Every service owns its persistence, answers synchronous reads over HTTP, and trades the high-frequency events over a broker.

		Express	
twin-service	Building Management	Node.js / Express	MongoDB
sensor-service	Telemetry Ingestion	Node.js / Express	MongoDB
contracts-service	Telemetry Distribution	Rust / Axum	MongoDB (read state cached in memory)
notification-service	Alerting	Node.js / Express	MongoDB
socket-service	Real-time transport	Node.js / Socket.IO	—
agent-service	Knowledge & Assistance	Python	PostgreSQL

Read more

The whole-system picture, the guiding principles and the routing table: Architecture Overview.

2.3 How the services talk

We use two styles of communication, and we chose between them on purpose:

- **Synchronous, over HTTP** through a single gateway — for the things a user does, and for the few service-to-service calls that genuinely need an answer on the spot. Identity rides along in the signed token, so no service ever has to phone another just to *authorise* a request.
- **Asynchronous, publish/subscribe over Redis** — for the steady stream of telemetry and alerts, where a producer should never be slowed down or blocked by whoever happens to be

pens — runs end to end across `sensor-service → contracts-service → socket-service`, while a second path raises threshold alerts through `notification-service` at the same time.

Read more

The two real-time sequence diagrams and the channel list: Communication & Data Flow.

2.4 What a service looks like inside

Zoom into most services and you find the same small, layered app: a strict **Router → Controller → Service → Model** flow, so anyone hopping between services already knows where the routing, the logic and the data access live. Three services break that pattern deliberately — the sensor service resolves a per-metric strategy at request time, the Rust contracts service keeps its hot state in memory (with its database behind it for durability), and the socket service is a flat broker-to-WebSocket bridge with no database at all. Authorisation and error handling we pulled out into shared middleware rather than copy-pasting them into every controller.

Read more

The layer responsibilities and the deliberate exceptions: Service Anatomy.

2.5 Keeping data consistent

Inside its own walls, each service is **strongly consistent** — it writes through its own database and that is that. Across services we settle for **eventual consistency**: a service finishes its local write, emits an event, and the others catch up asynchronously. No transaction ever stretches across two services. The handful of synchronous cross-service calls (used for provisioning) are best-effort and retried — never sitting on the hot path.

The database per service and stateless service rules that hold this together: Contributing › Architectural golden rules.

---

3 · DevOps

3.1 Building everything with one command

The developer experience stands on three layers, each with one job:

Layer	Its one job
just	The front door. Every workflow is a `just <recipe>` — short to type, easy to remember.
mise	The single source of truth for tool versions (Node, Python, Rust, uv, moon), identical on every machine.
moon	Per-project tasks — `build` / `test` / `lint` / `audit` / `deps` — with caching and "only what changed" detection.

The rule we never break: *don't trust whatever happens to be on your* `PATH`. Every tool is resolved through `mise`, and every package is declared exactly once in `.moon/workspace.yml`, with everything else — installs, cleans, the task graph — derived from that one list. That is what keeps a mixed TypeScript / Python / Rust build behaving like a single, tidy project instead of three.

How the three layers cooperate, and how adding a service is a one-line change: The Toolchain: mise, moon & just.

3.2 Containers, and where they run

Each service ships its own **two-stage Dockerfile** — one stage compiles, the final stage carries only the runtime and the built artifact — and refreshes its base-OS packages at build time so it clears the registry scan. The same images run in two places:

- **Docker Compose, on your laptop.** Each service's compose files are discovered and merged automatically. A service sits on a shared gateway network *and* a private database network, which enforces the database-per-service rule *physically* — the twin service simply cannot reach the auth database.
- **Kubernetes, in production.** Stateless services run as **Deployments**, the databases and Redis as **StatefulSets** with their own persistent volumes; an **nginx Ingress** routes by URL prefix, `initContainers` hold a service back until its database answers, and secrets are created on the cluster from a local `.env` and never go anywhere near Git.

Read more

Cluster manifests, ingress rewriting and rolling updates: Deployment & Kubernetes. The local compose layering: Docker Compose Orchestration.

3.3 Branching, and continuous deployment

We keep the branching model boring on purpose. There is **one long-lived branch,** `master`. All the real work happens on short feature branches, and the only way into `master` is a **pull request** — reviewed, and green on the CI gate. Direct pushes are blocked.

from — so the freshest `master` is also the freshest deployment. That is **continuous deployment**, and it is why the commit message is treated as a real input to the build rather than a note to ourselves.

Read more

The full workflow and the golden rules: Contributing & Golden Rules. Commit and branch conventions: Naming Conventions.

3.4 The pipeline

CI hangs off a **single aggregated gate** (`CI Gate / CI Passed`). One path-filter runs first, then the work fans out — in parallel, and *only for whatever actually changed* — to each service's checks (auth, chat, twin, sensor, socket, notification, contracts, client, agent) plus Docker, Kubernetes, workflow-lint and infra. The gate makes its call as a **whitelist**: every leg has to come back `success` or `skipped` , and the fan-out itself must have succeeded, so an unexpected state can never slip through as green.

Security isn't a separate stage — it's woven in:

- **Dependencies** — every service runs `npm` / `cargo` / `uv` audits, and a dependency-review leg fails the moment a PR *introduces* a vulnerable package.
- **Static analysis** — CodeQL (`security-extended`) over JavaScript/TypeScript, Python and Rust, on every PR, every push, and once a week.
- **Containers** — on `master` , images are built, **scanned with Trivy, and only pushed to GHCR if the scan passes** — never push first and scan later.
- **Supply chain** — every third-party action is pinned to a full commit SHA (enforced by a repo setting).

And the release itself runs without anyone touching it: every push to `master` runs **semantic-release** (work out the version → write the changelog → bump every manifest → tag → publish),

Read more

The gate’s decision logic, every leg, and the release/registry mechanics: CI/CD Pipeline → The CI gate and → CD workflows. The versioning rules: Commits & Semantic Release.

---

4 · Validation

Automation is useful only if the checks say something meaningful about the software. We therefore validate CrowdVision at more than one level: fast, deterministic tests protect each service while a pull request is still open; broader integration checks exercise the boundaries between services; and the agent has its own evaluation loop because a plausible sentence is not necessarily a correct, grounded answer.

4.1 Testing at the right level

Each stack uses the tools native to it rather than forcing the whole monorepo through one test runner. The Node services use **Jest**, the Vue client uses **Vitest**, the Rust contracts service runs its tests through **cargo llvm-cov**, and the Python agent uses **pytest** for both unit and integration tests. The client suite is split across three parallel runners, while moon’s affected-project detection keeps the backend fan-out focused on the services a pull request actually changed.

Those service-level tests are the everyday gate, but they are not the whole story. A composed backend integration suite exercises real service boundaries, and the frontend has **Playwright** end-to-end tests alongside manual cross-browser and API-contract checks. The slower, environment- dependent checks are kept outside the normal pull-request fan-out; the fast deterministic suites are the ones that must pass before a change can merge. This gives us quick feedback without pretending that an isolated unit test proves the entire distributed path.

The test runners, service workflows and local integration commands: CI/CD Pipeline → Service CI workflows · Running the Application → Testing.

4.2 Coverage is a signal, not a target

Every service measures coverage with its ecosystem's own instrumentation: Istanbul for Node and the client, LLVM coverage for Rust, and Cobertura output from pytest for Python. Pull requests expose the figures directly in their job summaries, and every push to `master` aggregates them into a single published summary and badge.

We deliberately **do not enforce a minimum percentage**. A universal threshold across a Vue interface, a message relay, a Rust hot path and an LLM-backed service would reward hitting a number more than testing the risky behaviour. Failed tests still block the merge; coverage instead tells the reviewer where confidence may be thin and where another test is worth asking for. The trade-off is explicit: coverage can reveal a regression, but the percentage alone never turns a red change green or a green change red.

Read more

The report formats, publication path and why the summary runs even after a failed test: CI/CD Pipeline → Code coverage.

4.3 LLMOps for the agent

The assistant needs a second kind of validation. Its deterministic shell — authentication, retrieval, citations and tool dispatch — is covered by the normal pytest suite, but the quality of a generated answer cannot be checked by comparing it with one fixed string. For that we maintain a **golden evaluation dataset** of hand-written questions paired with expected behaviour: which tool should run, which source should be cited, which facts should appear, and when the agent should say it does not know or refuse an out-of-scope request.

signals that the marker has become stale. The same runner can sweep several models, which lets us compare behaviour before changing the provider or default model.

These evaluations hit a real provider and an ingested corpus, so they are slower, cost money and are not hermetic. We run them on demand rather than disguising them as stable pull-request tests. In operation, **OpenTelemetry** traces exported to **Langfuse** complete the loop: each answer can be inspected as a tree of model calls, retrieval steps and tools, with latency, token use, cost and errors attached. Payload capture is off by default because prompts, documents and tool results may be sensitive; the structural trace remains available without them.

Read more

The evaluation dataset, model sweeps and trace design: Agent Service → Evaluation · → Langfuse Observability.

---

5 · A few decisions we are glad we made

Some choices shaped the *process* more than the product, and they're the ones worth lingering on.

- **Build, then scan, then push.** Running the vulnerability scan *before* the registry push — not after — means a flagged image is simply never published. Together with the SHA-pinned actions, it closes the supply-chain gaps that are easiest to miss.
- **A single list for the whole toolchain.** Because installs, cleans and the whole task graph all come from `moon query projects`, the package list cannot drift. Adding a service is a single line that then flows into build, test, lint, audit and install on its own.

Read more

The toolchain single source of truth and the Lerna call: The Toolchain · Contributing.

---

6 · Conclusion

Building CrowdVision was, more than anything, an exercise in **getting the process right first**. One polyglot build keeps Node, Python and Rust services behind a single interface; one CI gate checks every change with security woven through it; and a fully automated path carries a commit all the way from a pull request to a scanned image running on Kubernetes — with the architecture's boundaries kept in place by the tooling rather than by discipline alone.

Looking back, the process gave us four things we genuinely value:

- **A reproducible build** — one command, one cache, and no ambiguity about which tool version a given machine is using.
- **An automated path from commit to production** — versioning, changelog, image publishing and the rollout all happen without manual steps.
- **Security enforced by the gate** — dependency, static-analysis and container scans can block a merge or a publish, and the supply chain is pinned and tripwired.
- **A DDD model that held up in practice** — one bounded context per service, one database per service, kept honest by network isolation and by CI.

The full implementation depth is in the Developer Guide, the product walkthrough is in the User Guide; the HTTP contracts are in the API Reference.

Contents

1 · Introduction
- What we set out to build
- The exam constraints, and how we met them
- How we organised the work
- Licensing

2 · Design
- The domain, and how we split it
- Why each context became its own service
- How the services talk
- What a service looks like inside
- Keeping data consistent

3 · DevOps
- Building with one command
- Containers, and where they run
- Branching & continuous deployment
- The pipeline

4 · Validation
- Testing at the right level
- Coverage is a signal, not a target
- LLMOps for the agent

5 · Decisions we are glad we made

6 · Conclusion

Elsewhere

Developer Guide

On this page

CrowdVision — Software Process Engineering Report

1 · Introduction

- 1.1 What we set out to build
- 1.2 The exam constraints, and how we met them
- 1.3 How we organised the work
- 1.4 Licensing

2 · Design

- 2.1 The domain, and how we split it
- 2.2 Why each context became its own service
- 2.3 How the services talk
- 2.4 What a service looks like inside
- 2.5 Keeping data consistent

3 · DevOps

- 3.1 Building everything with one command
- 3.2 Containers, and where they run
- 3.3 Branching, and continuous deployment
- 3.4 The pipeline

4 · Validation

- 4.1 Testing at the right level
- 4.2 Coverage is a signal, not a target
- 4.3 LLMOps for the agent

5 · A few decisions we are glad we made

6 · Conclusion